

Mainrun Optimization Report

1. Objective

The objective of this project was to improve the validation performance of the provided GPT-2 style language model on the Hacker News headline dataset while complying with all assessment constraints. Throughout the optimization process, the dataset, random seed, number of training epochs, validation split, and evaluation procedure remained unchanged.

The provided baseline achieved a validation loss of 1.754. After a series of controlled optimization experiments, the final model achieved a validation loss of 1.1398, representing an absolute improvement of 0.6142 (approximately 35.0% lower validation loss than the provided baseline).

2. Baseline Reproduction

Before introducing any modifications, I reproduced the provided baseline implementation to establish a reliable reference point.

Running the original implementation without modification produced a final validation loss of 1.7518, closely matching the reported baseline of 1.754. This verified that the training pipeline was functioning correctly before beginning the optimization process.

3. Optimization Methodology

Rather than introducing multiple changes simultaneously, I adopted an incremental optimization strategy. Each experiment modified a single component while keeping all other settings fixed, allowing the impact of every change to be measured independently. Successful modifications were retained for subsequent experiments, while unsuccessful ones were reverted.

Across 42 experiments, the optimization process focused on four main areas:

- Training optimization
- Hyperparameter tuning
- Model architecture
- Dataset-specific inductive biases

This iterative approach made it possible to identify which techniques genuinely improved validation performance under the fixed seven-epoch training budget.

4. Training Optimization

4.1 AdamW Optimizer

The baseline model used SGD for optimization. Since optimization quality is often a major factor in Transformer training, I first focused on improving the optimizer before modifying the model architecture. AdamW is the standard optimizer for modern Transformer language models because it performs adaptive parameter updates while decoupling weight decay from the gradient update.

I replaced SGD with AdamW using a learning rate of $3e-4$, weight decay of 0.1, and β values of (0.9, 0.95). This significantly improved optimization stability and produced the largest improvement during the early stages of experimentation, reducing the validation loss from 1.7518 to 1.3071. The magnitude of this improvement suggested that the baseline was primarily optimization-limited, motivating further exploration of optimization-related hyperparameters before investigating architectural changes.

4.2 Learning Rate Tuning

After replacing SGD with AdamW, the validation loss improved substantially, indicating that optimization had become the primary bottleneck. I therefore investigated whether a different peak learning rate could make better use of the fixed seven-epoch training budget.

Multiple learning rates ranging from $2e-4$ to $1.2e-3$ were evaluated. Lower learning rates slowed convergence, while values above $1e-3$ produced diminishing returns without further improvement. The best performance was achieved using $1e-3$, which remained the final learning rate for the remainder of the project.

4.3 Warmup Scheduling

Increasing the peak learning rate also increased the risk of unstable optimization during the early stages of training. To improve stability while preserving a relatively large learning rate, I introduced a linear warmup phase before cosine learning-rate decay. Starting from a 5% warmup ratio, progressively longer warmup schedules were evaluated. Performance consistently improved until reaching 0.18, after which additional warmup reduced the effective optimization time available within the fixed seven epochs. Consequently, a 0.18 warmup ratio followed by cosine decay was selected as the final scheduling strategy.

4.4 AdamW β_2 Tuning

After the optimizer, scheduler, and architecture had largely stabilized, the remaining improvements became increasingly incremental. At this stage, I focused on fine-tuning AdamW itself by adjusting the second-moment decay coefficient β_2 , which controls how smoothly the optimizer estimates gradient variance.

Values of 0.95, 0.98, 0.99, 0.995, and 0.999 were evaluated. Each increase produced a small but consistent reduction in validation loss, with $\beta_2 = 0.999$ producing the best final result.

5. Regularization

5.1 Dropout

Throughout training, the validation loss continued decreasing until the final evaluation without showing clear signs of overfitting. This suggested that the model remained optimization-limited rather than regularization-limited under the fixed seven-epoch training budget.

Based on this observation, dropout was progressively reduced from 0.10 to 0.05, 0.02, and finally 0.00. Each reduction produced a small improvement for the original GELU-based architecture. After replacing GELU with SwiGLU later in the project, however, the larger feed-forward network benefited from additional regularization, and a dropout rate of 0.05 produced the best overall performance.

5.2 Weight Decay

Because reducing dropout consistently improved performance, I also investigated whether adjusting optimizer regularization would provide similar benefits. Weight decay values of 0.05, 0.10, 0.15, and 0.20 were evaluated.

Unlike dropout, reducing weight decay weakened regularization too much, while increasing it beyond 0.15 impaired optimization. A value of 0.15 therefore provided the best balance between optimization and generalization. I also evaluated selective weight decay, excluding LayerNorm and bias parameters, but this did not improve validation performance for this relatively small model.

6. Architecture Improvements

6.1 GPT-style Residual Initialization

Once the optimization procedure had largely stabilized, I shifted attention toward improving training dynamics through architectural modifications. One technique adopted by GPT-style models is scaling the initialization of residual projection layers to reduce the magnitude of residual updates during the early stages of training.

Following this approach, residual projection layers were initialized using

$$\frac{0.02}{\sqrt{2L}}$$

instead of the standard initialization. This produced more stable optimization and reduced validation loss while requiring only a minimal change to the existing implementation.

6.2 Rotary Positional Embeddings (RoPE)

After improving optimization stability, I investigated whether the positional encoding mechanism could be improved. The baseline model used learned absolute positional embeddings, whereas many recent Transformer architectures instead adopt Rotary Positional Embeddings (RoPE), which encode relative positional information directly within the attention mechanism.

Replacing learned positional embeddings with RoPE produced one of the largest architectural improvements observed during the project and remained part of the final model.

6.3 SwiGLU Feed-Forward Network

With the optimizer and positional encoding largely optimized, I next explored improving the representational capacity of the Transformer blocks themselves. SwiGLU has become a common replacement for GELU in modern language models because the gating mechanism often improves the effectiveness of the feed-forward network.

The initial implementation showed slight late-stage degradation, indicating that the larger feed-forward network required additional regularization. Introducing a small amount of dropout stabilized training and produced another substantial reduction in validation loss.

6.4 Transformer Architecture

Several model configurations with similar parameter counts were evaluated, including:

- 6 layers × 512 hidden size
- 8 layers × 448 hidden size
- 10 layers × 416 hidden size
- 10 layers × 448 hidden size

Although deeper models generally provided greater representational capacity, the ten-layer variants did not converge sufficiently within seven epochs. The 8-layer, 448-dimensional architecture provided the best balance between model capacity and optimization efficiency.

7. Dataset-specific Optimization

After exploring general optimization and architectural improvements, I looked for opportunities to better exploit the structure of the training dataset itself. Each Hacker News title represents an independent training example separated by an `<eos>` token, yet the baseline attention mechanism allowed tokens to attend across headline boundaries.

I modified the attention mechanism so that tokens could attend only to previous tokens within the same headline while preserving the standard causal mask. This introduced a dataset-specific inductive bias that prevented information leakage between unrelated titles.

This modification produced one of the largest improvements during the later optimization stages, reducing validation loss from 1.1648 to 1.1476, demonstrating that incorporating task-specific knowledge can be as effective as architectural improvements.

8. Additional Experiments

Not every modification improved performance. Several experiments were evaluated and subsequently reverted, including:

- selective weight decay
- vocabulary sizes of 12k and 20k
- smaller batch size
- deeper Transformer architectures
- RMSNorm

- PyTorch scaled dot-product attention (SDPA)
- cosine learning-rate floor

Although many of these techniques have been successful in larger language models, they did not improve validation performance under the constraints of this assessment.

9. Final Configuration

Component	Final Configuration
Transformer layers	8
Attention heads	8
Hidden dimension	448
Vocabulary size	16,000
Optimizer	AdamW
Learning rate	1e-3
Weight decay	0.15
AdamW β_1	0.9
AdamW β_2	0.999
Warmup ratio	0.18
Learning-rate schedule	Linear warmup + cosine decay
Feed-forward network	SwiGLU
Positional encoding	Rotary Positional Embeddings (RoPE)
Residual Initialization	GPT-style scaled initialization
Normalization	LayerNorm
Dropout	0.05
Attention masking	EOS-aware causal masking

10. Results

Model	Validation Loss
Provided baseline	1.7540

Reproduced baseline	1.7518
Final model	1.1398

Overall improvement:

- Absolute improvement: 0.6142
- Relative reduction in validation loss: 35.0%
- Total optimization experiments conducted: 42

11. Conclusion

This project explored 42 controlled optimization experiments covering optimizer selection, learning-rate scheduling, regularization, architectural modifications, initialization strategies, positional encoding, and dataset-specific attention mechanisms.

The largest improvements came from replacing SGD with AdamW, tuning the learning-rate schedule, adopting GPT-style residual initialization, introducing Rotary Positional Embeddings (RoPE), replacing the feed-forward network with SwiGLU, increasing model depth while maintaining a similar parameter budget, and preventing attention across <eos> boundaries.

The final model achieved a validation loss of 1.1398, substantially outperforming the provided baseline while remaining fully compliant with all assessment constraints. Throughout the project, emphasis was placed on systematic experimentation, reproducibility, and incremental validation to identify changes that consistently improved language model training.

Appendix A: Summary of Optimization Experiments

Trial	Modification	Validation Loss
00 (baseline)	None	1.7540
01	AdamW	1.3071
02	Linear warmup	1.2980
03-08	Learning-rate tuning	1.2455 (best)
09-11	Dropout tuning	1.2401 (best)
12-16	Weight decay tuning	1.2394 (best)
17-23	Warmup tuning	1.2237 (best)
24-26	Vocabulary & batch size	No improvement (x)
27	Residual initialization	1.2107

28	RoPE	1.1854
29-31	SwiGLU + dropout	1.1662 (best)
32-36	Architecture + normalization	1.1648 (best)
37	EOS-aware masking	1.1476
38-41	AdamW β_2 tuning	1.1398 (best)
42	Cosine LR floor	1.1413 (x)